Arquitectura De Integración

“Core Bank Integration Architecture”

Elaborado por: Wilian Orlando Nieves Rumipulla

Cuenca 08 de abril 2026

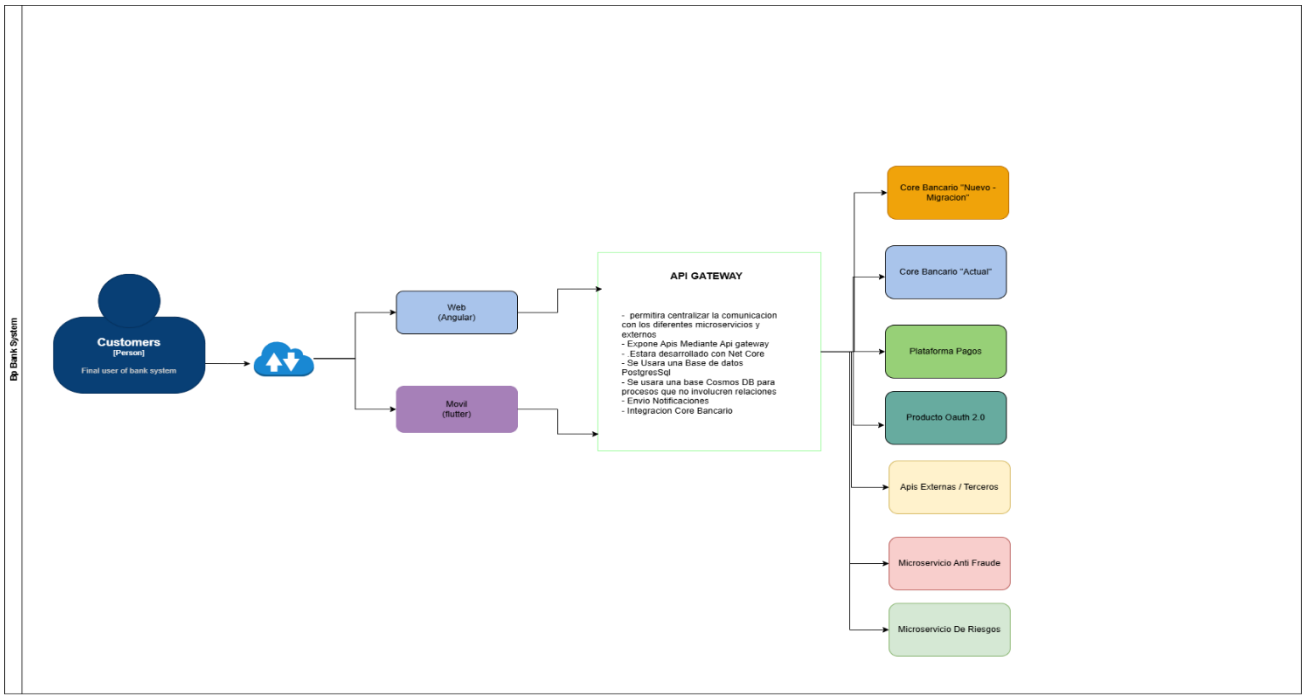
1. Introducción

Este documento contiene una propuesta de arquitectura de integración que permita la coexistencia de un core bancario antiguo con un nuevo core, habilitando canales digitales, integración con terceros, y capacidades avanzadas como gestión de riesgos y prevención de fraude.

La propuesta se basa en desacoplamiento, escalabilidad, resiliencia y cumplimiento normativo, utilizando una arquitectura orientada a microservicios, eventos y gobernanza de APIs.

2. Modelo De Arquitectura C4

2.1 Diagrama De Contexto



Se procedió a definir este modelo para la aplicación debido a que se requiere que sea web y a la vez que también tenga una parte móvil, para los usuarios finales se requiere que accedan mediante internet ya sea a la web o a la app móvil, adicional a esto se menciona usar una autenticación Oauth2 la cual ya la tienen y solo sería de implementar, es por esa razón que se definió separar el sistema con relación a uso de servicios o sistemas de proveedores externos.

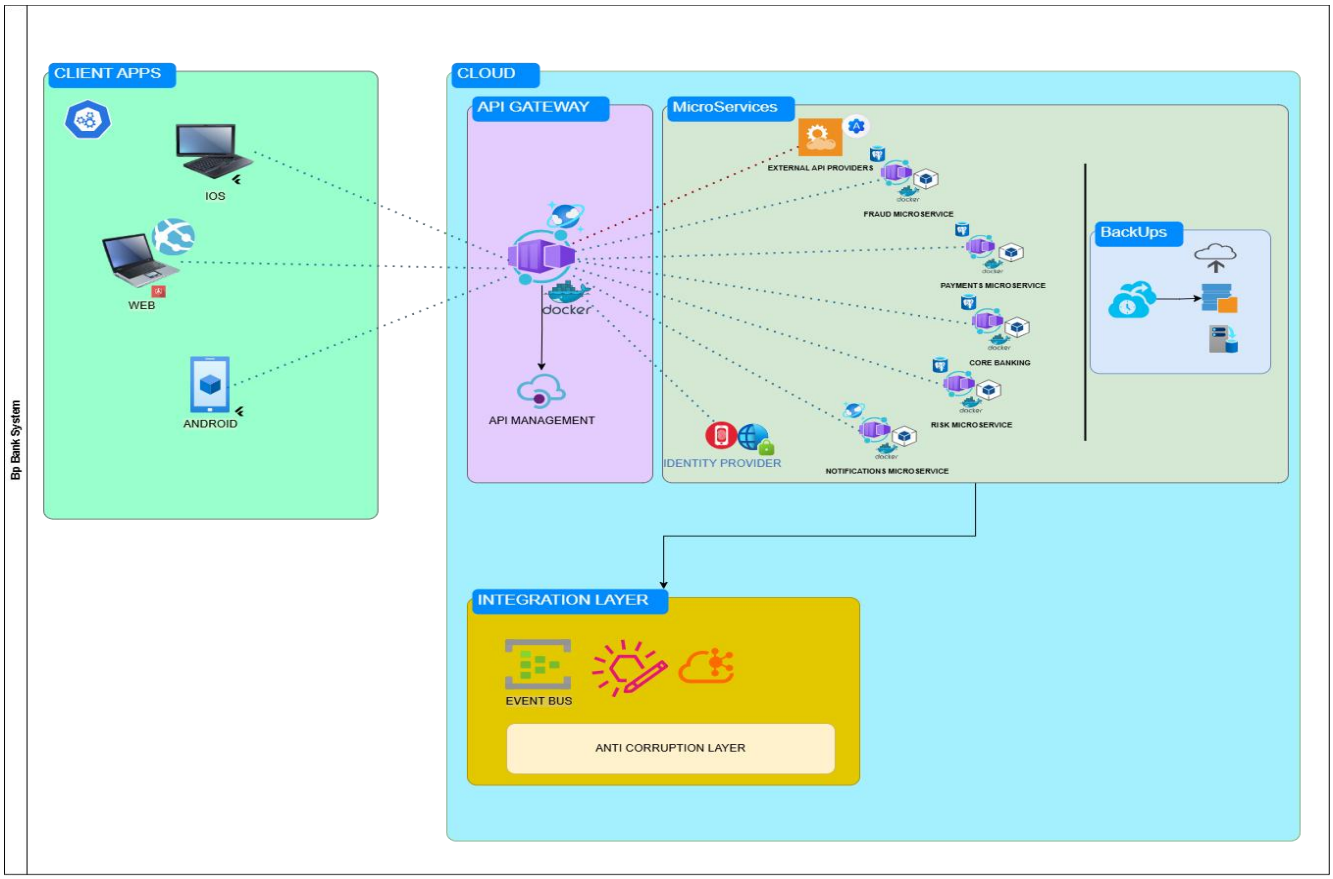
Estos dos clientes se conectaran al sistema bancario el cual tendrá expuestos los servicios que se requieran mediante un api Gateway el mismo que permitirá hacer uso de sistemas o servicios externos así como también permitirá hacer uso de servicios propios o que se vayan implementando en el core bancario según la necesidad del cliente,adicional tendremos un servicio de notificaciones el cual podrá ir implementando diferentes tipos de notificaciones según se requiera para los usuarios esto será transparente ya que los sistemas o servicios solicitados están relacionados

pero no son dependientes entre si lo cual garantizara la comunicación entre las diferentes áreas del negocio.

Se analizo un modelo unificado, pero no se opto por ese modelo ya que con el tiempo puede volverse inmantenible y no garantizaría la versatilidad y adaptabilidad al cambio de acuerdo con las exigencias del negocio.

Se define un ecosistema donde el sistema bancario se expone a través de un API Gateway, permitiendo la integración controlada con las diferentes integraciones y comunicación entre microservicios como con externos. Se incorpora una capa de integración que desacopla el core legacy mediante patrones de anti-corrupción, permitiendo evolucionar hacia un core digital sin afectar la operación actual.

2.2 Diagrama De Contenedores



Para este diagrama se definió para el usuario final tener una interfaz web desplegada en app service de azure usando angular y para el tema móvil se propone usar flutter, se analizó la viabilidad de usar react y react native sin embargo se opta por angular ya que es un framework robusto y adicional se analiza que al recurso humano para la adaptación al uso de angular tiene una curva de aprendizaje menor a react, en el tema móvil se opta por flutter porque no requiere cambios significativos si se requiere

desarrollar para ios como lo es react native ya que las dependencias hay que probarlas e inclusive ir implementándolas de acuerdo al sistema para el que se está desarrollando.

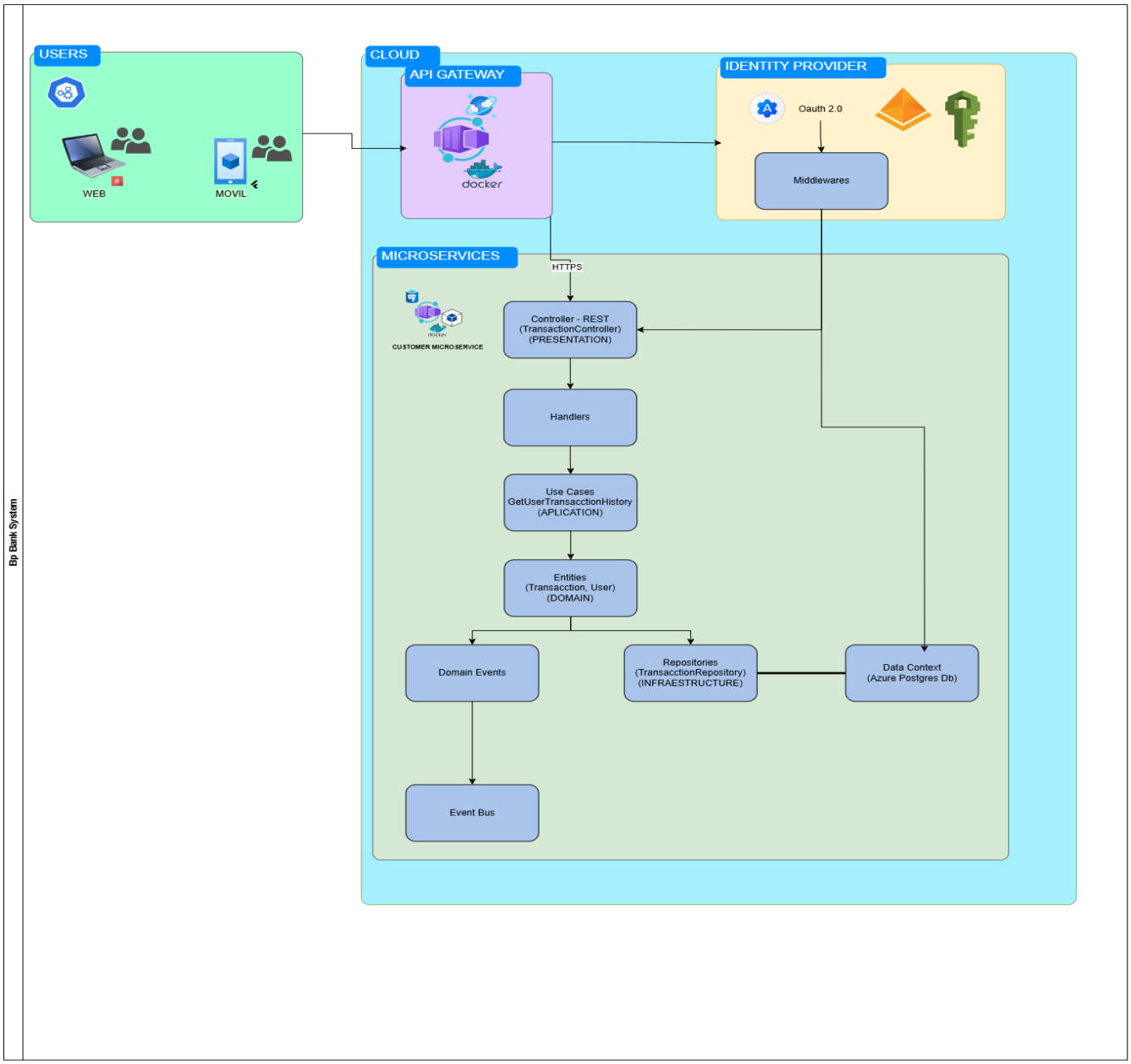
Para el tema de arquitectura se opta por usar microservicios desacoplados mediante la implementación de eventos y la implementación de un api Gateway para temas de seguridad, escalabilidad, mantenibilidad, adicional a tener todo centralizado y tener un mejor control de acceso como un punto de entrada único, los lenguajes que se definen para el api Gateway como para los microservicios son .net debido a que es un lenguaje adaptable y versátil, así como también puede representar rapidez para el usuario. Se analizo trabajar bajo una arquitectura monolito sin embargo de acuerdo con lo solicitado y con el constante cambio en el sector bancario este no sería mantenible, al trabajar con microservicios yo puedo separar responsabilidades y trabajar sobre servicios de manera independiente y también porque al requerir realizar productos que se puedan reutilizar para otros sistemas aplica perfectamente al uso de microservicios lo cual no se podría al trabajar con una arquitectura monolito.
En cuanto a lenguajes se analizó trabajar con Python ya que es super rápido al trabajar con un orm como django o flask sin embargo por el tema de crecimiento y escalabilidad que se requiere este no aplica ya que no está diseñado para trabajar con microservicios.

En temas de base de datos se usará un postgres y cosmos como servicios de azure, postgres para microservicios como customermicroservice que podría tener una relación interna, y cosmos para microservicios como notificationMicroservice ya que solo se requiere hacer un envió de notificaciones y registrar el mismo como también se podría usar para servicios de reporteria a futuro.

Se opta por azure ya que es propio de Microsoft, así como .Net lo cual facilita algunas funciones que se podrían aprovechar.

Dentro de esta arquitectura la comunicación síncrona se gestionaría mediante APIs y la asíncrona mediante un bus de eventos.

2.3 Diagrama De Componentes



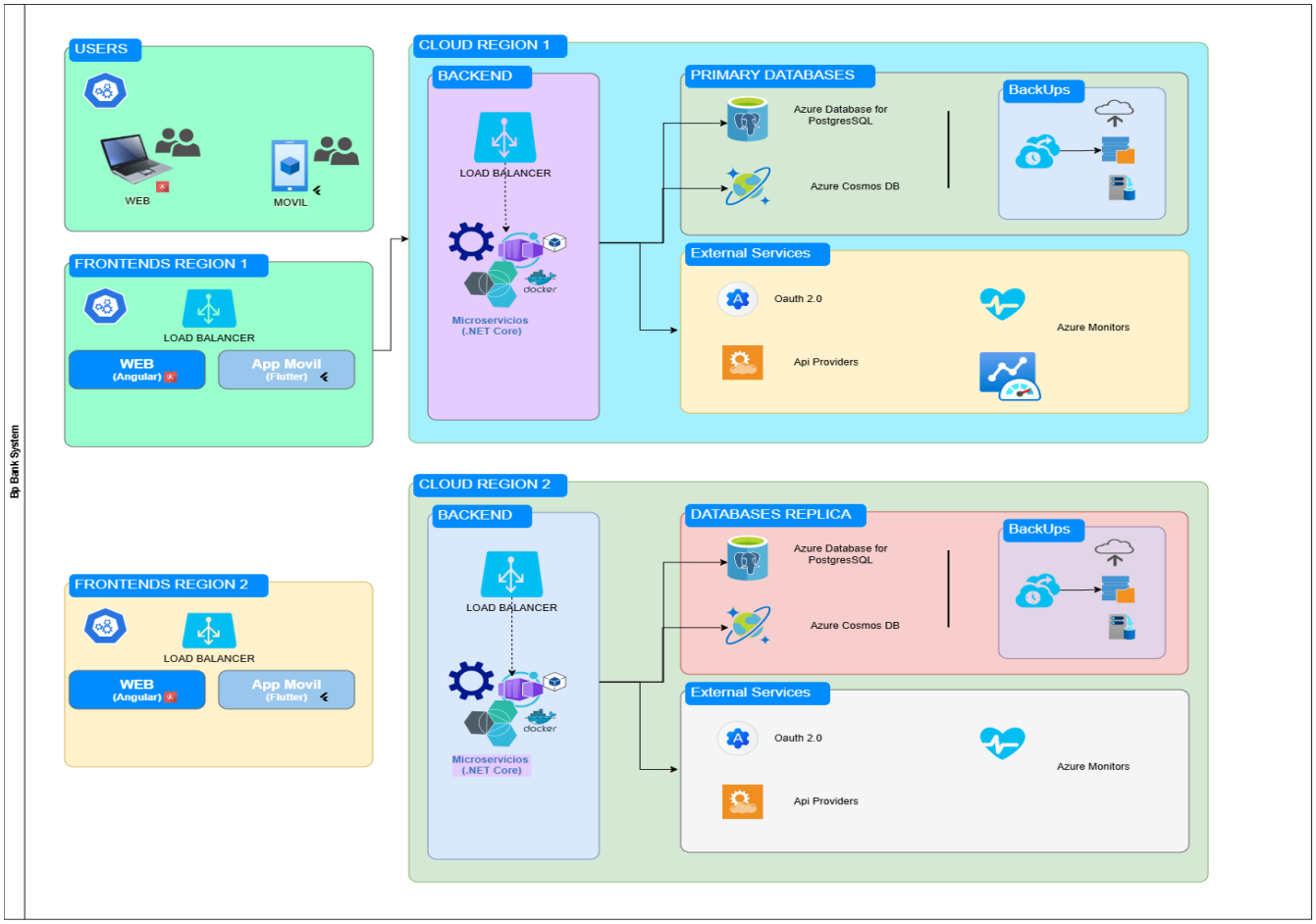
En este diagrama como se puede identificar se aplicó el uso de clean architecture para separar las capas de dominio, aplicación e infraestructura ya que esto permite garantizar el tema de mantenibilidad a largo plazo, también se puede identificar que todas las peticiones se receptan vía https y con una autenticación Oauth2 para garantizar que solo las peticiones que cumplan con esta autenticación puedan acceder a los servicios o información necesaria.

También se optó por clean architecture ya que permite separar y delegar el tema de responsabilidades sobre las capas mencionadas y una vez cumpla con los procesos necesarios procede a comunicarse con las bases de datos en este microservicio explícitamente con una postgres de azure pero cabe recalcar que de acuerdo con el microservicio este podría comunicarse también con el servicio de azure de cosmos db. Por último, se definió usar clean architecture ya que esta arquitectura facilita el tema de test, consistencia entre servicios y también se puede adaptar de mejor manera a cambios tecnológicos.

Se analizo el trabajar con mvc pero no es recomendable para este proyecto debido a la magnitud de este y también por el tema de dependencias ya que al realizar cambios en base de datos o la lógica del negocio esto en varias ocasiones genera conflictos directos en otras capas y porque este patrón esta más orientado a proyectos monolitos.

En cuanto al tema de patrones se propone implementar el patrón cQRS para separar operaciones de lectura y escritura. Con esto se optimizaría rendimiento y escalabilidad y también se propone hacer uso de eventos de dominio para separar procesos críticos como pagos y validaciones antifraude estos se encaminarían a hacer uso de un bus de eventos.

2.4 Diagrama De Despliegue



Como se puede evidenciar en el diagrama los despliegues de toda la arquitectura del proyecto se realizara en azure para aprovechar los servicios administrados y para garantizar disponibilidad y tener una mejor tolerancia a fallos, también se implementará monitores propios de azure para tener un panorama claro de lo que pasa con los recursos o si presentan fallos y que nos permita actuar a tiempo ante estos escenarios.

se plantea hacer uso de container apps para los despliegues del api Gateway como para los microservicios que estarán bajo dockerizacion, esto con la finalidad de poder realizar despliegues continuos que no generen impacto sobre los usuarios finales ya que al configurar esto, las modificaciones que se desplieguen no serán percetibles para los clientes.

Para temas de despliegue continuo se prevé trabajar con la configuración de workflows que trabajen con github actions lo cual permite interactuar con los repositorios de dockerhub y reduce tiempo para el personal humano ya que el responsable se podrá centrar en la revisión de código, aplicación de buenas practicas y test, una ve cumpla con esto procederá con la fusión de ramas y commits respectivos.

Por ultimo cabe mencionar que al realizar despliegues en nube en este caso azure, nos permite tener un gran nivel de estabilidad y escalabilidad en tema de recursos que garantizaran la continuidad del negocio, en cuanto a redes también en azure hay una versatilidad en temas de enlaces y redundancia pero este punto si se basa en la configuración que se vaya dando a cada uno de los recursos y cabe mencionar que al hacer uso de container apps ante un fallo el levantamiento de un nuevo recurso es cuestión de minutos ya que este podría estar enlazado a un mismo App environment y un log analitycs workspace para tema de logs.

Se analiza realizar un despliegue en un ambiente on-premise pero no es recomendable por temas de escalabilidad en cuanto a recursos como capacidad de cpu,ram o almacenamiento, mientras que en azure permite una escalabilidad dinámica adicionalmente si se opta por un on-premise se tiene que tener muy en cuenta temas de licenciamiento, costos , aplicación de parches de seguridad y administración de seguridad física, mientras que en azure como se mencionaba existen servicios adminitrados además de contar con certificaciones que requiere el sector bancari como la iso 270001.

También se analizó trabajar con AWS pero con la estructura y tecnologías propuestas para microservicios se requiere mayo configuración mientras que en azure podemos tener una mejor integración ya que son ecosistemas propios de Microsoft.

3. Modelo De Dominio

Se Adoptaria el estándar BIAN ya que este permite estructurar arquitecturas en dominios funcionales, facilitando la reutilización y evolución independiente de cada capa y en este caso ayudaría a definir capacidades del negocio bancario de forma desacoplada, por ejemplo:

Dominios principales:

- Customer Management
- Account Management
- Payments
- Fraud Detection
- Risk Management

4. Patrones De Integración

Los patrones que se detallan en el documento permiten construir una arquitectura flexible, tolerante a fallos y alineada con las necesidades de un sistema bancario y se detallan a continuación:

- **API Gateway**
  Punto de entrada único para consumidores internos y externos.
- **Event-Driven Architecture**
  Uso de eventos para desacoplar servicios y mejorar resiliencia.
- **CQRS**
  Separación de lectura y escritura.
- **Anti-Corruption Layer**
  Protección del core bancario evitando acoplamiento directo y fraudes.

5. Seguridad y Cumplimiento

Para el tema de seguridad para el accesos a equipos y plataformas se aplicaría OAuth2 + OpenID Connect y Identity Provider (Azure AD B2C), que hace referencia a un modelo modelo Zero Trust donde cada servicio valida identidad y permisos en cada interacción.

Con respecto a la protección de datos aplicaría encriptación tls y encriptación en reposo, adicionalmente se debería implementar zero trust architecture y el uso de api security, con esto se estaría cumpliendo con las regulaciones financieras, así como con la protección de datos personales.

Adicional a esto para lo que respecta a monitoreo se debería implementar un log centralizado, configuración de alertas automatizadas, revisión de métricas en cuanto a recursos como “CPU y latencia”

6. Alta Disponibilidad y Recuperación ante Desastres

Para garantizar la continuidad del negocio la estrategia se propone realizar despliegues multirregión y replicaciones de bases de dato mediante redundancia geográfica y para la recuperación ante desastres se aplicaría el uso de mecanismos de recuperación automática como backups o failover automáticos.

7. Plan De Migracion

Para lo que respecta a la migración esta debería realizarse de manera incremental para minimizar los riesgos operativos y también deberíamos validar cada fase de migración. Para la ejecución se define las siguientes fases:

- Implementación API Gateway
- Implementación Capa De Integración
- Migración de funcionalidades críticas “Pagos, clientes”
- Implementación de eventos
- Desacoplamiento progresivo del Core bancario